

Arquitectura de Computadores

Arquitectura de computadores - Semana 1 y 2

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Conocer los aspectos fundamentales de la arquitectura de un computador multinivel y la evaluación de su rendimiento

Específicos

- Definir y delimitar el concepto de arquitectura.
- Explorar la perspectiva multinivel en el ámbito de la arquitectura.
- Analizar la Arquitectura von Neumann.
- Identificar los aspectos clave para incrementar el rendimiento de un computador.
- Clasificar los tipos de arquitecturas paralelas.
- Examinar las áreas estratégicas relevantes dentro del campo de la arquitectura.

Contexto de Arquitectura de Computadores



Contexto de Arquitectura de Computadores

Computador Digital

Máquina capaz de aceptar unos datos de entrada, efectuar con ellos operaciones lógicas y aritméticas y proporcionar la información resultante a través de un medio de salida de forma automática.

Arquitectura de Computadores

La arquitectura de computadoras se refiere a la estructura interna y diseño de los componentes de un sistema informático. Engloba el conjunto de instrucciones, la organización de la memoria, la interconexión entre unidades y el paralelismo. Una buena arquitectura busca optimizar el rendimiento, mejorando la velocidad y eficiencia en el procesamiento de datos y tareas.

Contexto de Arquitectura de Computadores

¿Qué más es Arquitectura de Computadores?

- Evoluciona constantemente para mejorar el **rendimiento**, la eficiencia energética y la capacidad de procesamiento de las computadoras.
- El rendimiento se logra mediante un **equilibrio entre el hardware y el software**, permitiendo un uso más eficiente de los recursos y una respuesta más rápida a las demandas de la persona usuaria.
- Los avances en esta área han sido fundamentales para **impulsar el desarrollo tecnológico y mejorar la experiencia del usuario** en una amplia gama de aplicaciones, desde dispositivos móviles y computadoras personales hasta supercomputadoras y sistemas embebidos en diferentes industrias.

Contexto de Arquitectura de Computadores

La evolución de computadores está influenciada

Causas físicas

- **Desarrollo de tecnologías clave**, desde los primitivos relés y válvulas de vacío de las primeras generaciones, pasando por los transistores y circuitos integrados de las posteriores, hasta los microprocesadores y nanotecnología de la actualidad.
- Estos avances permiten la creación de computadoras cada vez más rápidas, potentes y eficientes, transformando la capacidad de procesamiento y almacenamiento.

Causas no físicas

- La **reorganización** de los componentes internos y la **lógica de funcionamiento** ha sido esencial para mejorar la eficiencia y rendimiento junto con el diseño en arquitecturas más sofisticadas.
- Estas mejoras han facilitado la interacción entre la persona usuaria y el computador, permitiendo una programación más amigable y eficiente, así como una experiencia de uso más intuitiva y funcional.

Niveles de descripción de un computador



Niveles de descripción de un computador

Estrategia de estudio de cualquier sistema complejos

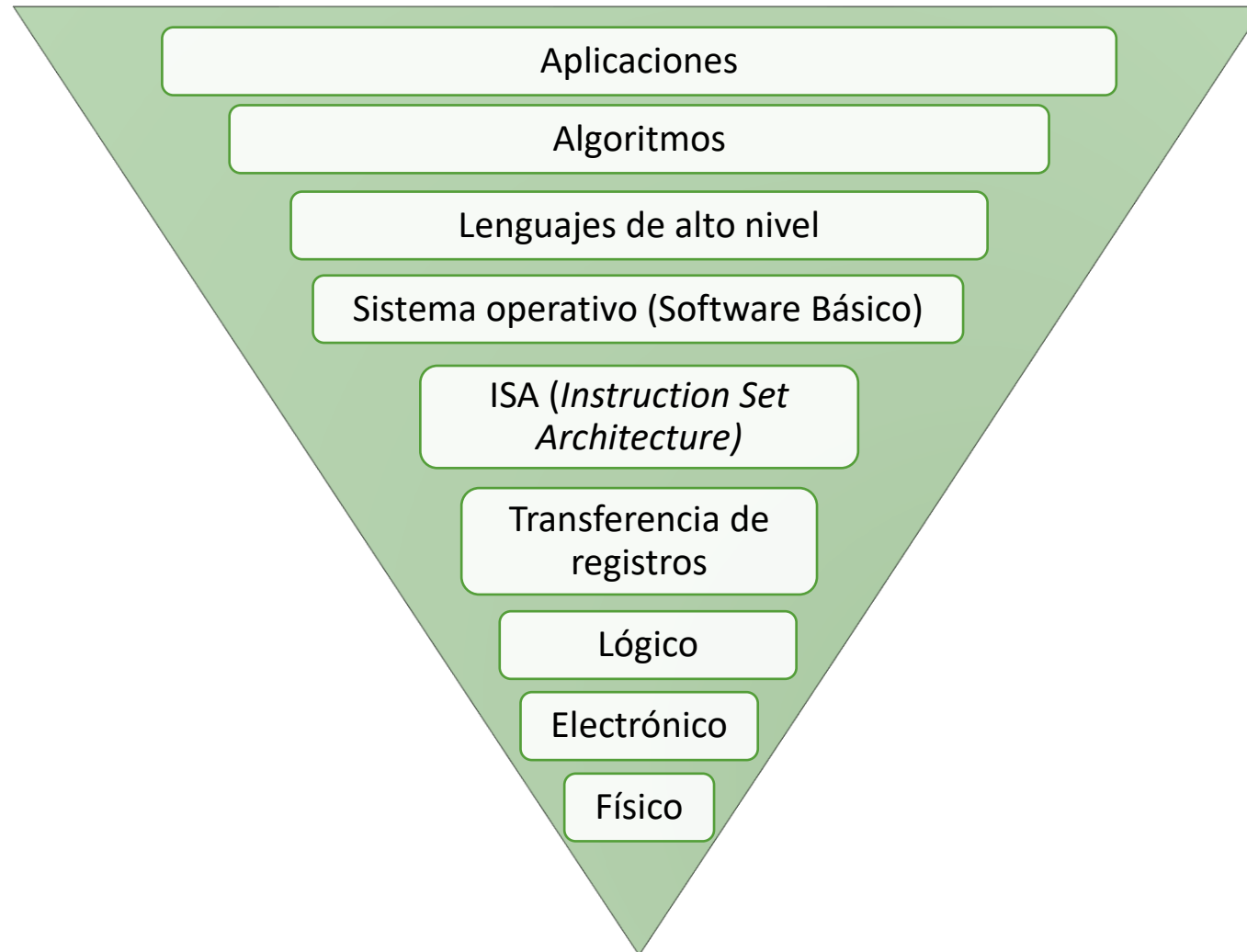
Estructurar el sistema en diferentes niveles de abstracción para que la complejidad quede dividida, acotada y organizada en cada nivel, en el cual se puede aplicar una metodología propia de estudio.

Cada nivel de abstracción se caracteriza por:

- Unos elementos de entrada disponibles para el diseño en este nivel, y que proceden del nivel inmediato inferior.
- Unos elementos de salida, es decir, objetivos del diseño en este nivel, y destinados al nivel inmediato superior.
- Una metodología de análisis y síntesis de los elementos de salida en términos de los de entrada.

Niveles de descripción de un computador

Niveles de abstracción en el estudio de un computador



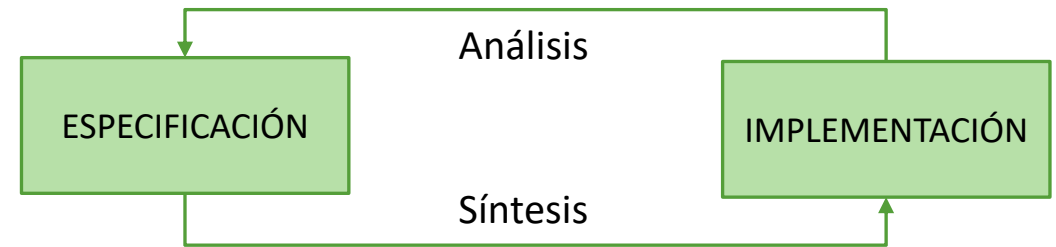
Procesos de estudio: análisis y síntesis

Análisis

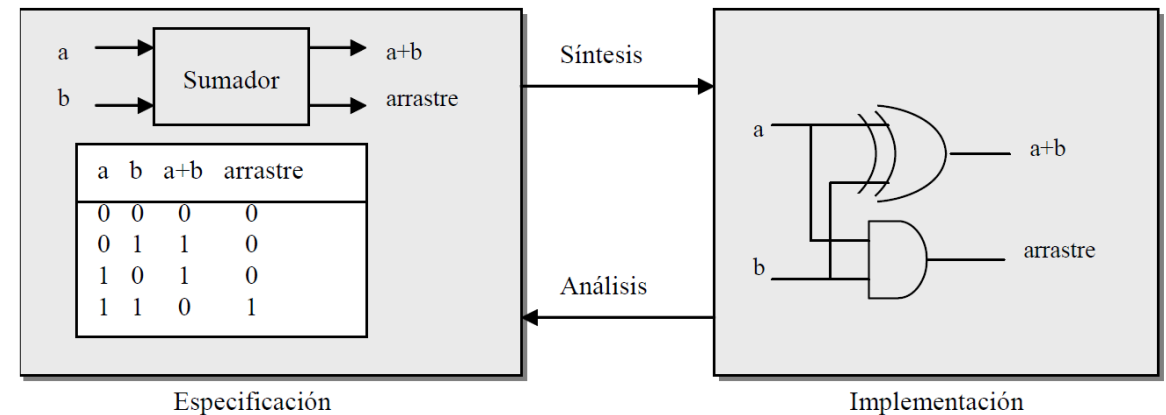
Parte de la implementación del sistema del nivel inferior y determina la función del sistema en dicho nivel, es decir, su **especificación**.

Síntesis

Parte de la especificación de un sistema y obtiene su **implementación** en función de los elementos básicos del nivel inferior.



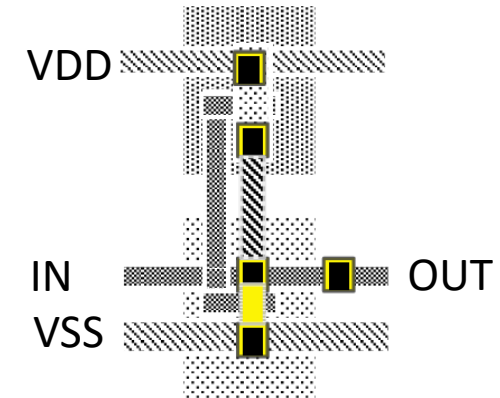
Ejemplo:



Niveles de descripción de un computador

Nivel Físico

- Se manipulan como elementos de entrada las formas geométricas que se corresponden con las máscaras de difusión utilizadas en el proceso de fabricación de los circuitos integrados del computador.
- Determinadas disposiciones de estas formas representan **dispositivos electrónicos** concretos, tales como transistores, resistencias, etc., que son los elementos de salida del nivel físico.
- En este nivel se suele utilizar como herramienta de estudio software de manipulación gráfica con restricciones.



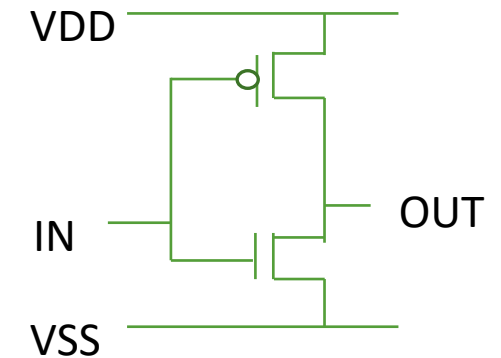
Físico (silicio)

- Las formas geométricas superpuestas representan las diferentes capas de fabricación usadas en el proceso de integración: difusión, polisilicio, contactos metálicos, etc.
 - IN: Entrada de datos o señales al chip.
 - OUT: Salida de datos o señales desde el chip.
 - VSS: Conexión a tierra o voltaje negativo .
 - VDD: Alimentación de voltaje positivo para el chip.
- La disposición de estas capas permite implementar físicamente el transistor y conectarlo con el resto del circuito.

Niveles de descripción de un computador

Nivel Electrónico

- En este nivel los elementos de salida, es decir, **los biestables y las puertas lógicas**, se obtienen a partir de dispositivos electrónicos (resistencias, transistores, etc.) conectados según una determinada topología.
- Como metodología de estudio (análisis y síntesis) en este nivel se utilizan técnicas cuantitativas de análisis en el plano eléctrico-temporal, fundamentalmente **ecuaciones algebraicas y diferenciales**.



Circuito eléctrico

Niveles de descripción de un computador

Nivel Lógico

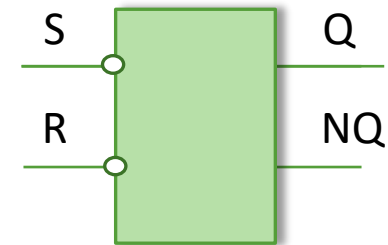
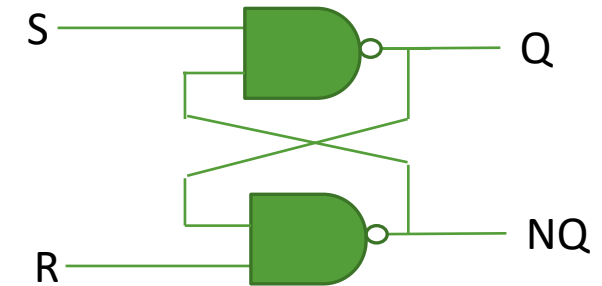
- Los elementos de entrada a este nivel son los biestables y las puertas lógicas, y los de salida son **módulos combinacionales y secuenciales** tales como multiplexores, codificadores, sumadores, registros, contadores, etc.
- Este nivel de estudio dispone de sus propias técnicas de análisis y síntesis. El comportamiento de un circuito combinacional se representa con una función lógica que admite una expresión algebraica manipulable simbólicamente dentro de un formalismo matemático: **el álgebra de conmutación** (un álgebra de Boole). Para los circuitos secuenciales se utiliza la teoría de las **máquinas de estados finitos**.

Niveles de descripción de un computador

Nivel Lógico

Ejemplo

- Un circuito de ejemplo es el mostrado arriba corresponde a un biestable SR implementado con compuertas NAND, un elemento secuencial básico que permite almacenar un bit de información mediante dos estados estables (Q y NQ)
- S (Set): ordena poner la salida Q en "1".
- R (Reset): ordena poner la salida Q en "0"
- Abajo se encuentra el circuito simplificado



Niveles de descripción de un computador

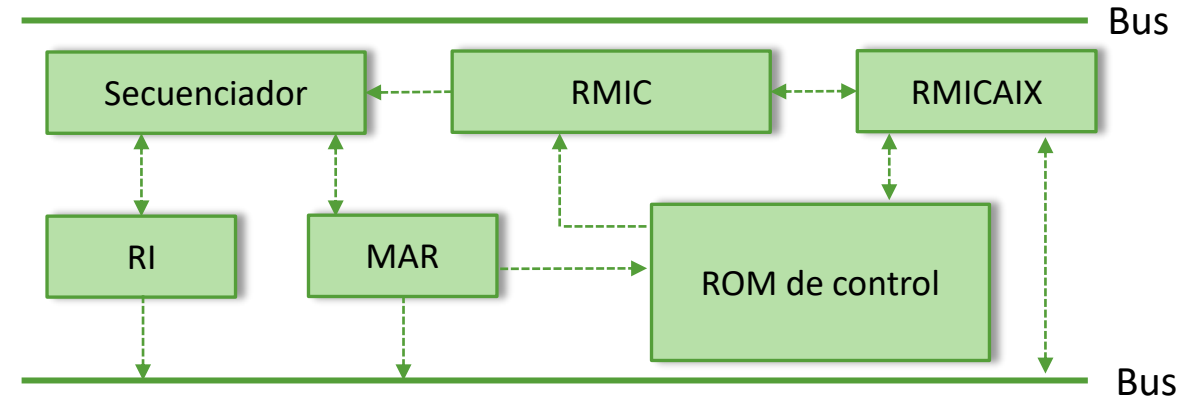
Transferencia de Registros (RT)

- Los elementos de entrada al nivel RT son registros, módulos combinacionales y elementos de interconexión (buses y/o multiplexores).
- Los registros mantienen el estado del sistema, los módulos combinacionales definen las transformaciones elementales del estado, y los buses y/o multiplexores permiten el intercambio de información entre los dos primeros.
- Los elementos de salida son el conjunto de transferencias elementales posibles en la ruta de datos construida con los tres tipos de elementos de entrada.
- Es posible utilizar los modernos lenguajes de descripción hardware (por ejemplo VHDL) como instrumentos de expresión precisa y estándar.

Niveles de descripción de un computador

Transferencia de Registros (RT)

- **Secuenciador:** Genera la secuencia de microinstrucciones que debe ejecutar la unidad de control. Se encarga de determinar cuál es la siguiente microinstrucción según las condiciones actuales.
- **RI (Registro de Instrucción):** Contiene la instrucción de máquina actual que ha sido cargada desde memoria. De aquí se extraen los campos de operación y direcciones.
- **MAR (Memory Address Register / Registro de Dirección de Memoria):** Almacena la dirección de memoria que se va a leer o escribir.
- **ROM de Control:** Es una memoria de solo lectura que almacena las microinstrucciones que implementan el conjunto de instrucciones del procesador.
- **RMIC (Registro de Microinstrucción de Control):** Guarda la microinstrucción actual que se está ejecutando. Sus campos generan las señales de control para el procesador.
- **RMICAUX (Registro Auxiliar de Microinstrucción de Control):** Sirve como apoyo al RMIC, normalmente para almacenar microinstrucciones condicionales, de salto o de apoyo al secuenciador.



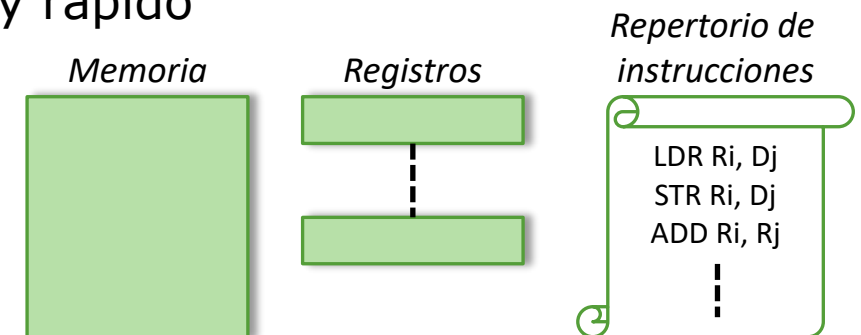
Flujo básico en el esquema

- El RI recibe la instrucción desde memoria a través del bus.
- El MAR guarda direcciones que permiten localizar la microinstrucción correspondiente en la ROM de Control.
- La ROM de Control proporciona la microinstrucción, que se carga en el RMIC.
- El Secuenciador decide cuál será la siguiente microinstrucción y coordina la transferencia entre RMIC, RMICAUX y la ROM de Control.

Niveles de descripción de un computador

Instruction set architecture (ISA):

- Elementos de entrada: transferencias y transformaciones posibles de información en la ruta de datos.
- Elementos de salida: lenguaje máquina definido por el repertorio de instrucciones (formatos, direccionamiento, etc.) y la memoria y conjunto de registros referenciables por las instrucciones.
- Metodología de estudio: tecnología e ingeniería del software, existiendo dos planteamientos
 - CISC (*Complex Instruction Set Computer*): define un repertorio de instrucciones complejo y numeroso, con muchos tipos de direccionamiento y modos de control.
 - RISC (*Reduced Instruction Set Computer*): simplifica la complejidad y el número de instrucciones, dejándolo reducido a un conjunto pequeño y rápido



Niveles de descripción de un computador

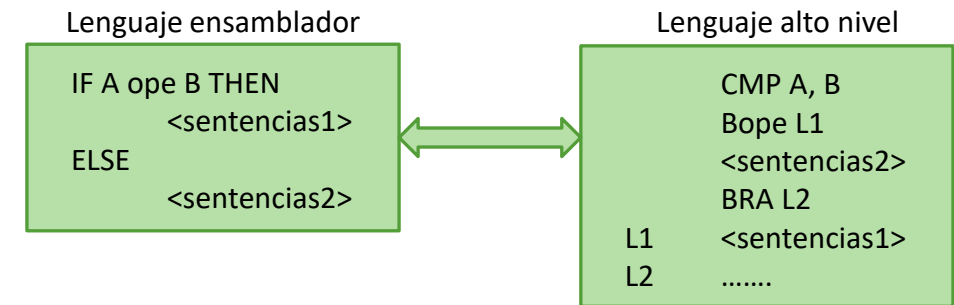
Software básico (sistema operativo)

- El Sistema Operativo (SO) no constituye un nivel del mismo tipo que los demás. Se trata de un gestor de determinados recursos del nivel máquina que por la frecuencia y complejidad de uso resulta más eficiente utilizarlos de modo conjunto.
- Las funciones del SO en los primeros computadores eran escasas, limitadas básicamente a la carga del programa y a la entrada salida.
- Las competencias en las funciones del SO en las máquinas modernas han ido aumentando, englobando entornos multiusuario y multitarea, por lo que requieren una gestión de todos los recursos de la máquina:
 - CPU, jerarquía de memoria, tratamiento de las excepciones, mecanismos de protección, gestión de archivos, etc.
- El protagonismo que el SO tiene en un computador actual hace que **cuando se diseña la arquitectura** de un procesador se tengan muy en cuenta las funciones de este sistema.

Niveles de descripción de un computador

Lenguaje de alto nivel

- En este nivel se utilizan lenguajes de programación con una sintaxis y una semántica más complejas que las del lenguaje ensamblador.
- Facilitan el trabajo del programador aportando recursos expresivos más próximos a los problemas que se van a resolver.
- Se requiere un proceso de traducción al nivel máquina que es realizado por un programa denominado compilador.



- Existen dos paradigmas de lenguajes de alto nivel:
 - Lenguajes imperativos (Java, C++, C, etc.) con una semántica operacional que obliga al programador a expresar la secuencia de órdenes cuya ejecución resuelve el problema.
 - Lenguajes declarativos (Prolog, SQL o Lisp) con una semántica que permite que el programador tan solo declare las relaciones lógicas o funcionales del problema.

Niveles de descripción de un computador

Algoritmos

- En el nivel algorítmico se expresa la **resolución de un problema** mediante un conjunto de reglas aplicadas de forma sistemática y ordenada.
- Los procedimientos que define un algoritmo son independientes de cualquier lenguaje de programación y de cualquier máquina particular.

Aplicaciones

- Las aplicaciones se corresponden con **dominios de actividad** que pueden automatizarse con el uso de un computador digital.
- Del análisis del dominio se extraen unas **especificaciones funcionales** que son expresadas mediante algoritmos.
- Codificados estos algoritmos en un lenguaje de programación y previa compilación, se ejecutan en la máquina.

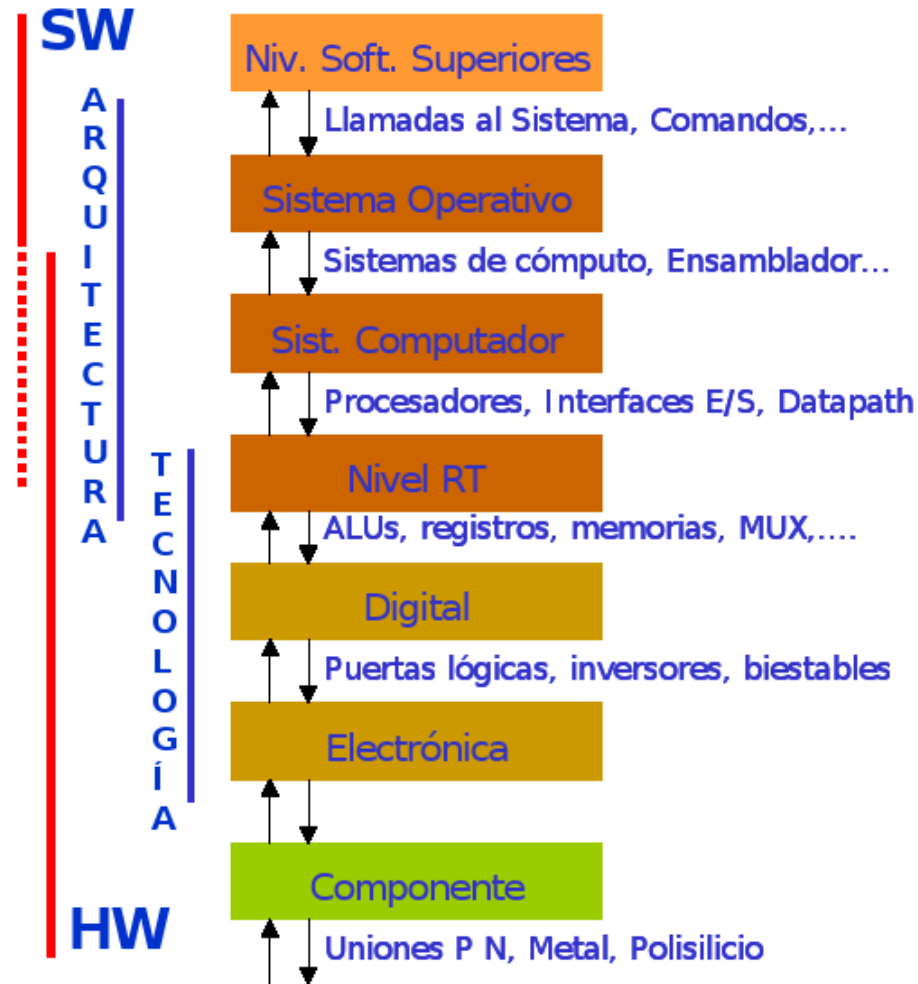
Niveles de descripción de un computador

Inconveniente de descripción por niveles

- A la hora de implementar una especificación no sólo deben cumplirse todos los requerimientos funcionales de la misma. Además se deben **optimizar ciertas funciones de calidad** que, generalmente, tienen que ver con la velocidad (maximización) y el costo (minimización).
- Existen ocasiones en las que contemplar tan solo los niveles frontera de un nivel en el que se plantea un problema de diseño puede dar lugar a la imposibilidad de optimizar la implementación.

Niveles de descripción de un computador

Otra descripción por niveles



ORTEGA, J.; ANGUIA, M. Arquitectura de Computadores en seis créditos ECTS. *Enseñanza y Aprendizaje de Ingeniería de Computadores*, 2012, vol. 2012, no 2, p. 13.

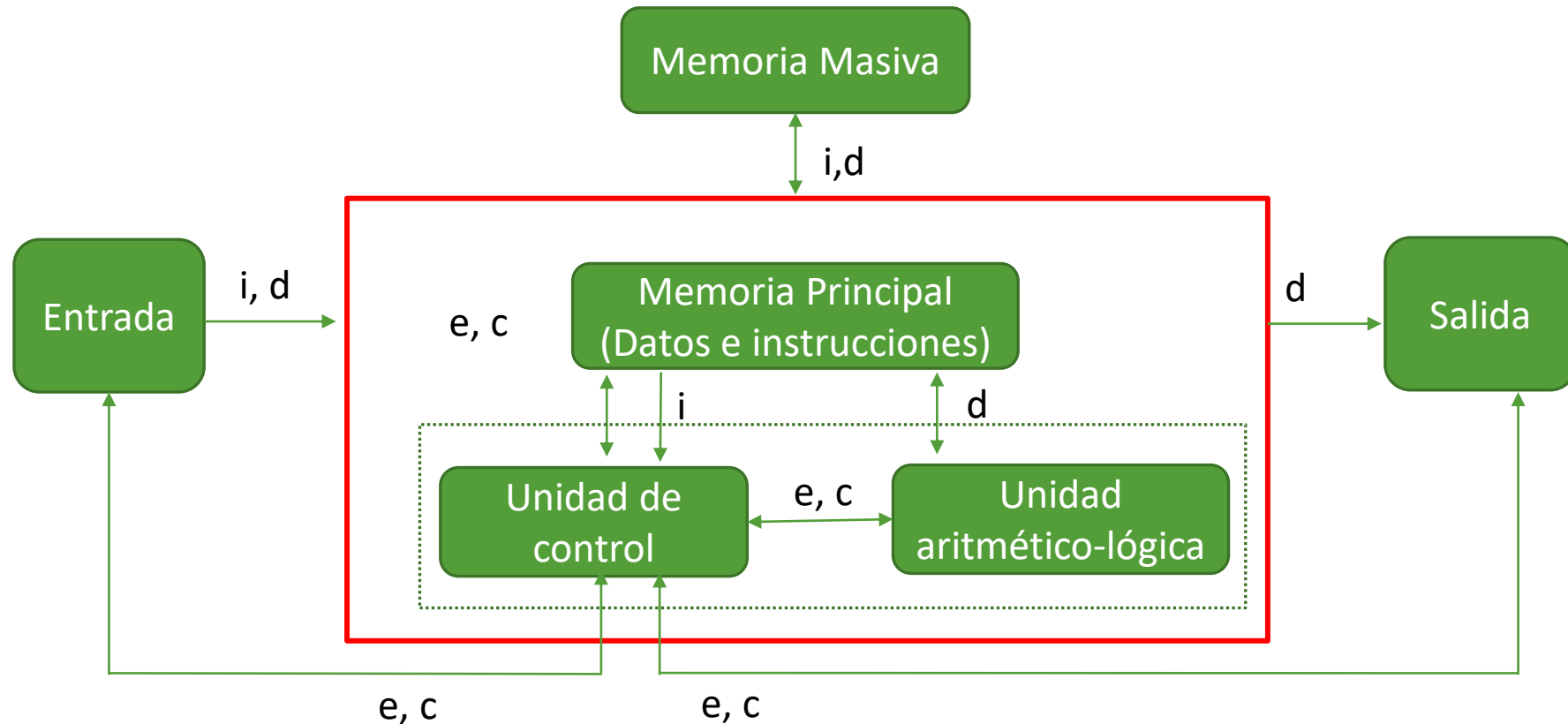
Arquitectura von Neumann



Arquitectura von Neumann

Estructura básica de un computador

La estructura básica de un computador actual está basada en la estructura original de John von Neumann, una máquina secuencial que ejecuta datos escalares.



i: flujo instrucciones
d: flujo de datos
c: señales de control
e: señales de estado

Arquitectura von Neumann

Unidades

- La **memoria** almacena las instrucciones del programa, los datos iniciales, los resultados parciales y los finales.
- El **procesador (CPU)** es la unidad encargada de leer y ejecutar las instrucciones. Para ello dispone de una ruta de datos constituida por un conjunto de registros, una unidad aritmético-lógica, y unos buses de comunicación. Además, una unidad de control es la encargada de generar las señales que gobiernan todos los dispositivos.
- La **entrada y salida** constituyen la unidad para la transferencia de información con el exterior.

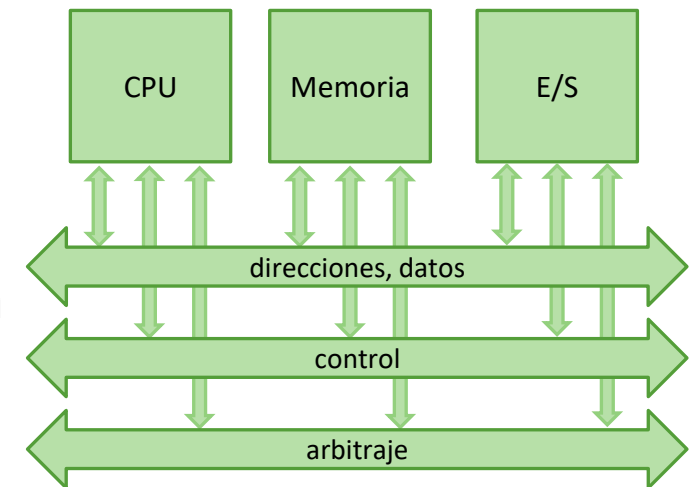
Características de la máquina

- Organización lineal de la memoria.
- Palabra de longitud fija.
- Espacio único de direcciones.
- Memoria única para datos e instrucciones sin diferenciar entre ambos.
- Ejecución secuencial de las instrucciones salvo las de ruptura de secuencia.

Arquitectura von Neumann

Comunicaciones

- Un bus es un medio compartido de comunicación constituido por un **conjunto de líneas** (conductores) que conecta las diferentes unidades de un computador.
- La principal función de un bus es servir de soporte para la realización de **transferencias de información** entre dichas unidades.
- La unidad que inicia y controla la transferencia se conoce como *master* del bus para dicha transferencia, y la unidad sobre la que se realiza la transferencia se conoce como *slave*.
- Los papeles de *master* y *slave* son dinámicos, de manera que una misma unidad puede realizar ambas funciones en transferencias diferentes.
- Para garantizar el **acceso ordenado al bus**, existe un sistema de arbitraje que establece las prioridades cuando dos o más unidades pretenden acceder al mismo tiempo al bus.
- Para establecer el tiempo de duración de las transferencias y que sea conocido tanto por el *master* como por el *slave*, un bus debe disponer de los medios necesarios para la **sincronización master-slave**.



Arquitectura von Neumann

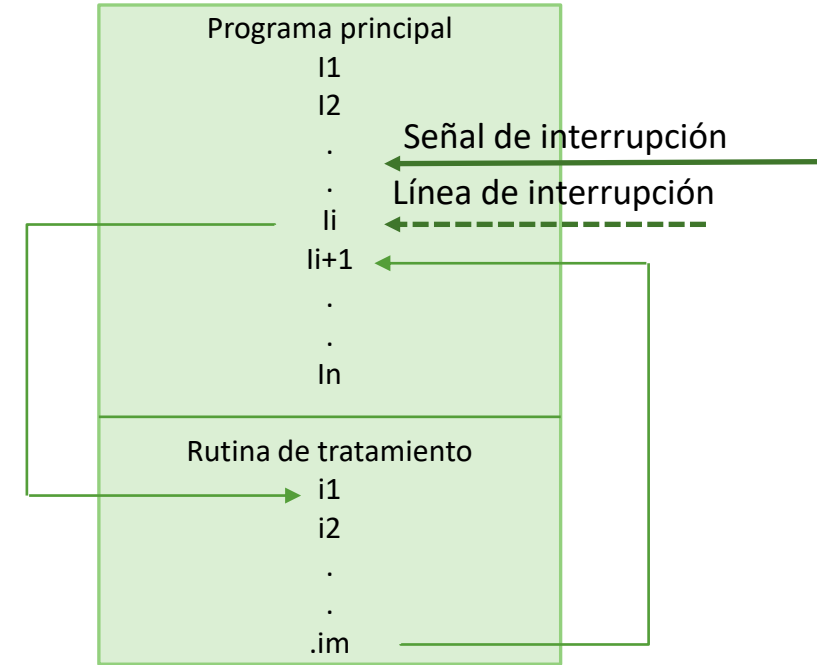
Comunicaciones

- **Líneas de información básica.** Las utiliza el *master* para definir los dos elementos principales de una transferencia, el *slave* y los datos. Se dividen:
 - Líneas de Direcciones. Determinan la unidad que hace de *slave* en la transferencia
 - Líneas de Datos. Transportan los datos de la transferencia.
- **Líneas de control.** Transmiten las órdenes que determinan la operación de transferencia a realizar por las líneas de datos y direcciones, y marcan el ordenamiento temporal de las señales que circulan por el bus.
 - Líneas de control propiamente dichas: escritura en memoria, lectura de memoria, operación de salida, operación de entrada, etc.
 - Líneas de sincronismo: reconocimiento de transferencia, reloj, reset, etc.
- **Líneas de arbitraje.** Establecen la prioridad entre diferentes peticiones de acceso al bus: petición del bus, cesión del bus y ocupación del bus.

Arquitectura von Neumann

Sistema de interrupciones

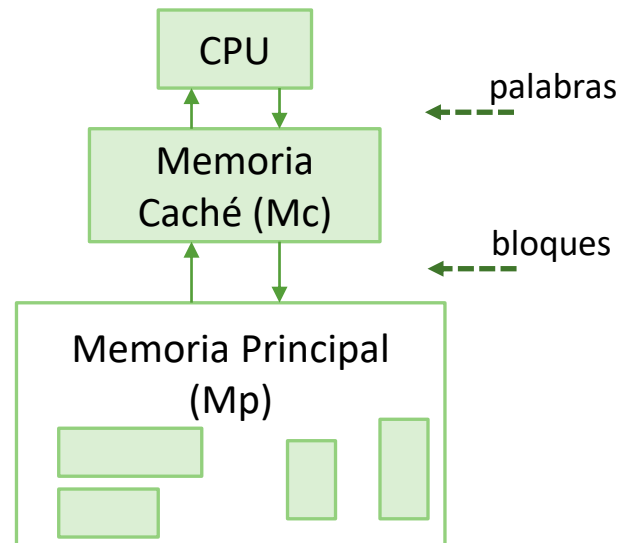
- El sistema de interrupciones permite que la CPU detenga la ejecución del programa principal cuando recibe una señal externa o interna que requiere atención inmediata.
- El procesador ejecuta el programa principal de forma secuencial (I1, I2, ..., Ii).
- Al producirse una interrupción, se interrumpe temporalmente la ejecución normal.
- La CPU transfiere el control a una rutina de tratamiento de interrupción, que atiende el evento (i1, i2, ..., im).
- Finalizada la rutina, la CPU retoma el programa principal en la instrucción siguiente (Ii+1).
- Con este mecanismo se logra una mejor sincronización con los dispositivos de Entrada/Salida y se facilita el uso compartido de la CPU por múltiples programas, aumentando la eficiencia del sistema.



Arquitectura von Neumann

Sistema de memoria caché

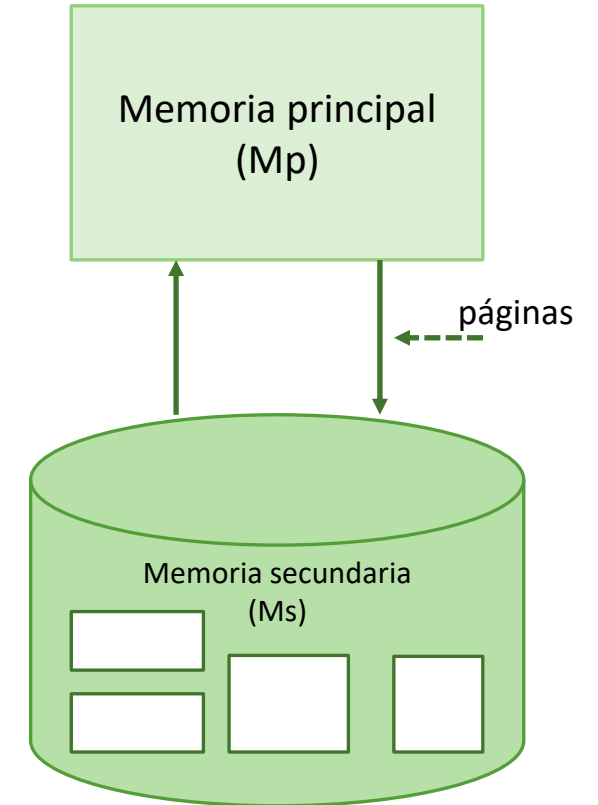
- El Sistema de memoria caché permite disminuir el tiempo de acceso a la memoria principal (Mp) ubicando una memoria de menor tamaño y mayor velocidad (memoria caché, Mc) entre la CPU y Mp.
- El sistema explota la localidad de referencia de los programas haciendo que Mc contenga en cada momento los bloques de Mp más referenciados, y evitando así que la CPU tenga que acceder a Mp.



Arquitectura von Neumann

Sistema de memoria virtual

- El Sistema de memoria virtual permite la ejecución de programas cuyo tamaño supere el de la Memoria Principal.
- Para ello el sistema mantiene en Memoria Principal, de forma transparente para el programador, tan sólo el conjunto de páginas activas (con mayor probabilidad de ser referenciadas) del programa en ejecución.
- Las restantes páginas que completan el programa residen en la memoria secundaria, hasta que son referenciadas, en cuyo caso el sistema las activa llevándolas a Memoria Principal.



Paralelismo



Paralelismo

Descripción

Las posibles formas de dotar de paralelismo a un sistema monoprocesador basado en la arquitectura Von Neumann son:

- Multiplicidad de unidades funcionales
- Segmentación encauzada (*pipelining*)
- Solapamiento de operaciones de Entrada y Salida (E/S) y CPU
- Equilibrado de los anchos de banda de los subsistemas
- T_{es} (Tiempo de entrada y salida) $> T_m$ (Tiempo de memoria) $> T_p$ (Tiempo de procesador)
- Multiprogramación y tiempo compartido

Paralelismo

Tipos de paralelismo

Paralelismo funcional

Aquel que se refleja en la lógica de la solución de un problema (en los diagramas de flujo y, por tanto, en el código). Puede ser explotado por la arquitectura y/o por el sistema operativo (SO)

- A nivel de instrucción (grano fino). Solamente por la Arquitectura
- A nivel de bucle o hebra (grano medio). Arquitectura y SO
- A nivel de procedimiento (grano medio). Arquitectura y SO
- A nivel de programa (grano grueso). Solamente por el SO

Paralelismo de datos

Se consigue mediante el uso de estructuras de datos que permiten operaciones paralelas sobre sus elementos (vectores, matrices...)

Paralelismo a nivel de arquitecturas

Clasificación de arquitecturas paralelas. Taxonomía de Flynn: se basa en el número de unidades de control y de procesadores disponibles en el computador.

- Flujo de instrucciones simple (SI): la arquitectura tiene una única unidad de control que produce un único flujo de instrucciones
- Flujo de instrucciones múltiple (MI): la arquitectura tiene múltiples unidades de control, cada una produciendo un flujo distinto de instrucciones
- Flujo de datos simple (SD): sólo hay un procesador que ejecuta un único flujo de datos
- Flujo de datos múltiple (MD): hay varios procesadores, cada uno de ellos ejecutando un flujo de datos distinto.

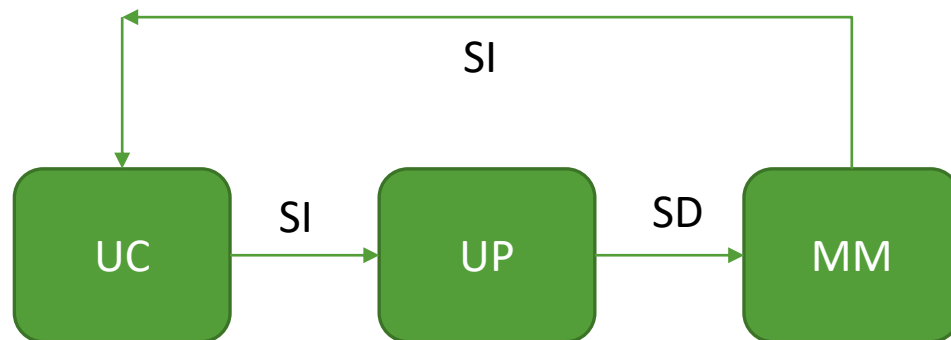
	SI	MI
SD	SISD	MISD
MD	SIMD	MIMD

Arquitectura	Paralelismo Funcional	Paralelismo Datos
SISD	Segmentados, superescalares y VLIW	
SIMD		Vectoriales y Matriciales
MIMD	Multiprocesadores, Multicomputadores y Clusters	

Paralelismo a nivel de arquitecturas

SISD

- Representa a la mayoría de los ordenadores actuales.
- Las instrucciones se ejecutan secuencialmente, aunque sus etapas de ejecución pueden estar solapadas.
- Un computador SISD puede tener más de una unidad funcional, aunque todas bajo la supervisión de una única unidad de control.

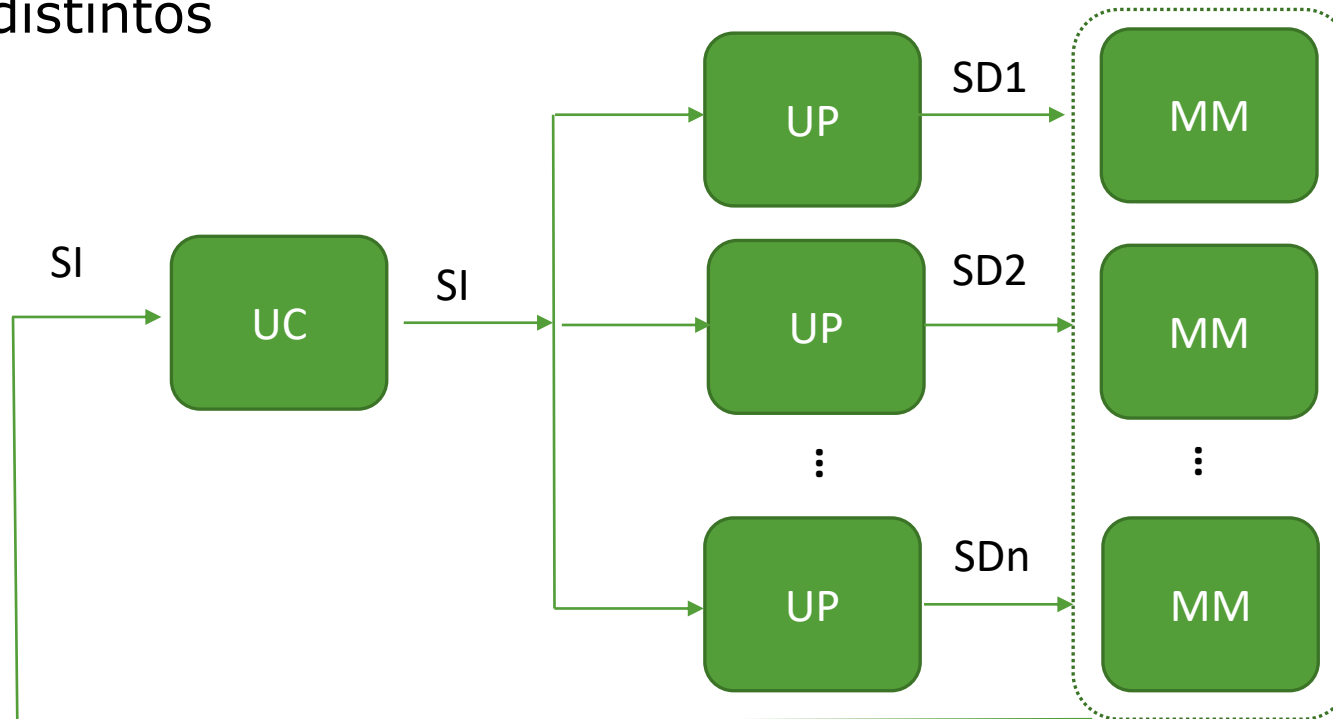


UC: Unidad de control
UP: Unidad de proceso
MM: Módulo de Memoria
SI: Flujo individual de instrucción
SD: Flujo individual de datos

Paralelismo a nivel de arquitecturas

SIMD

- Se corresponde con los ordenadores vectoriales y matriciales
- Hay múltiples unidades de proceso, supervisadas por una única unidad de control
- Todos los procesadores reciben la misma instrucción, pero cada uno opera sobre datos distintos

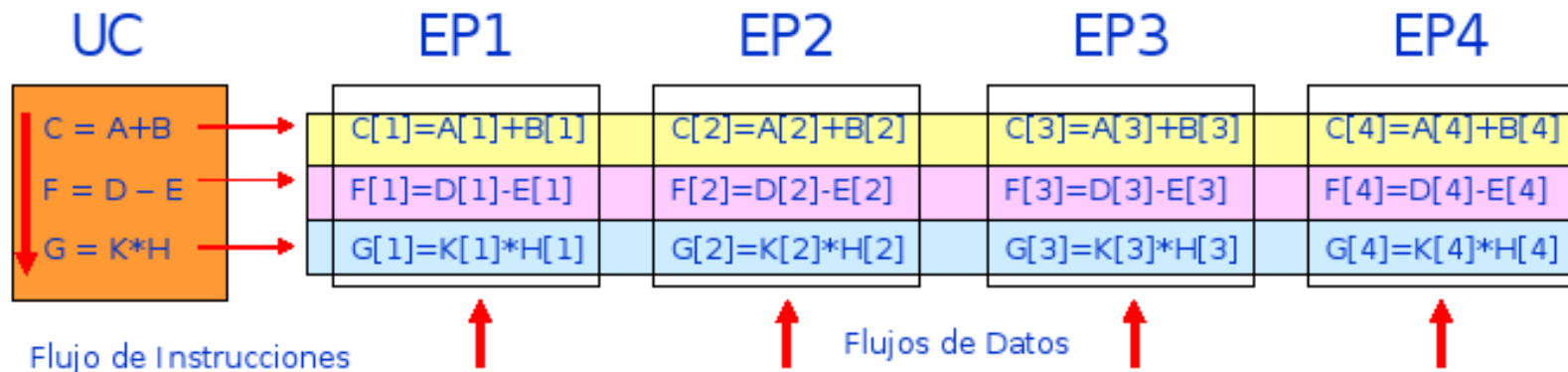


UC: Unidad de control
UP: Unidad de proceso
MM: Módulo de Memoria
SI: Flujo individual de instrucción
SD: Flujo individual de datos

Paralelismo a nivel de arquitecturas

SIMD

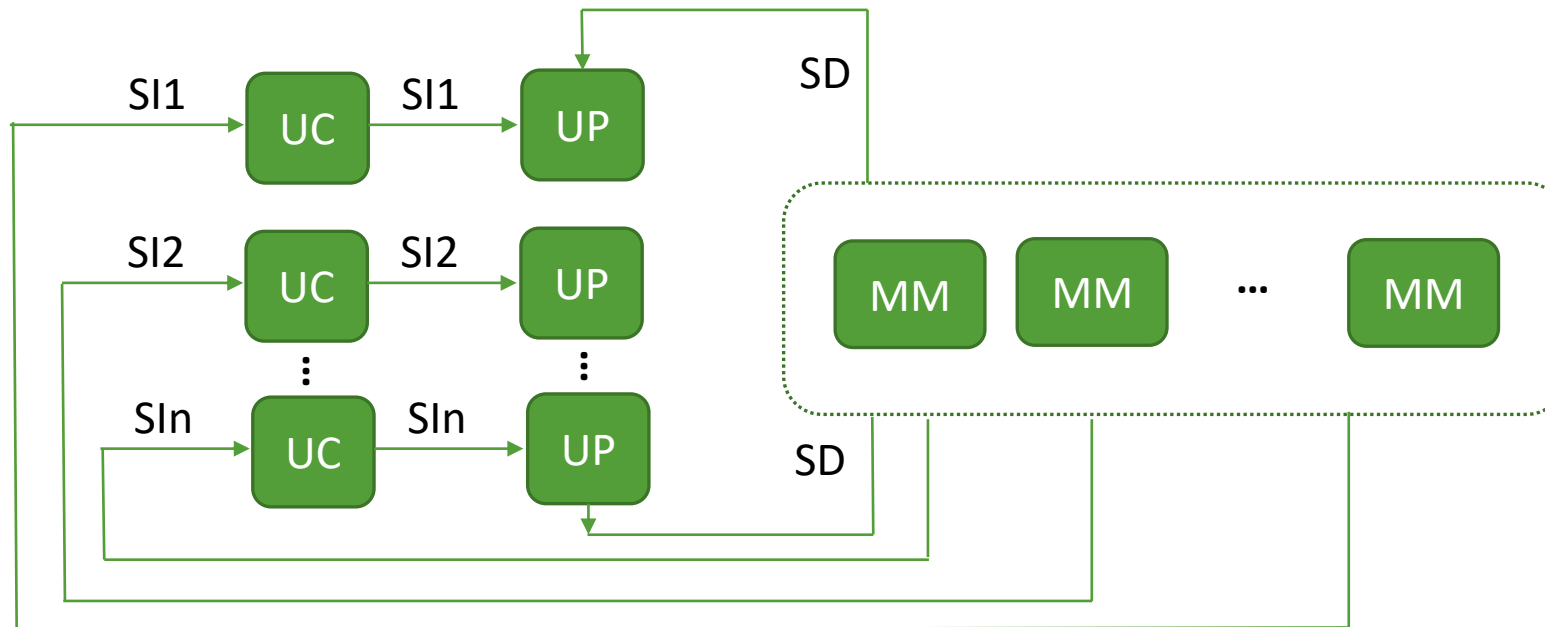
- Una unidad de control proporciona una secuencia de instrucciones que se ejecutan sincronamente operando sobre distintos flujos de datos
- Requieren menos memoria puesto que ejecutan el mismo código.
- Apropriados para problemas que presenten mayor paralelismo de datos
- Todos los procesadores ejecutan el mismo código de forma síncrona: habrá procesadores trabajando y desocupados, según los datos.
- El diseño de los programas es más sencillo.



Paralelismo a nivel de arquitecturas

MISD

- Hay n procesadores, cada uno recibe una instrucción diferente pero operan sobre el mismo conjunto de datos.
- La salida de un procesador es la entrada del siguiente.
- No hay ninguna máquina comercial que siga esta arquitectura.

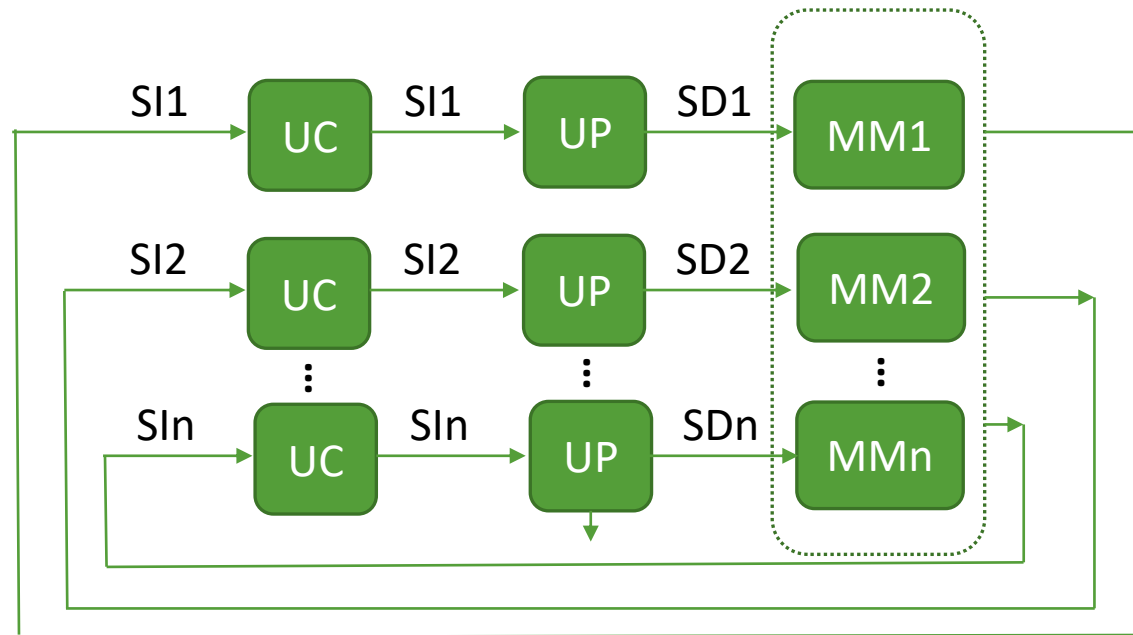


UC: Unidad de control
UP: Unidad de proceso
MM: Módulo de Memoria
SI: Flujo individual de instrucción
SD: Flujo individual de datos

Paralelismo a nivel de arquitecturas

MIMD

- En los MIMD cada unidad de procesamiento funciona independientemente y puede ejecutar sus propios programas de forma asíncrona
- Los MIMD presentan una mayor escalabilidad puesto que se pueden agregar distintas unidades de procesamiento. Hay dos tipos: memoria compartida (multiprocesadores) y memoria distribuida (multicomputadores)



UC: Unidad de control
UP: Unidad de proceso
MM: Módulo de Memoria
SI: Flujo individual de instrucción
SD: Flujo individual de datos

Rendimiento y mejoras en prestaciones



Rendimiento

Computador

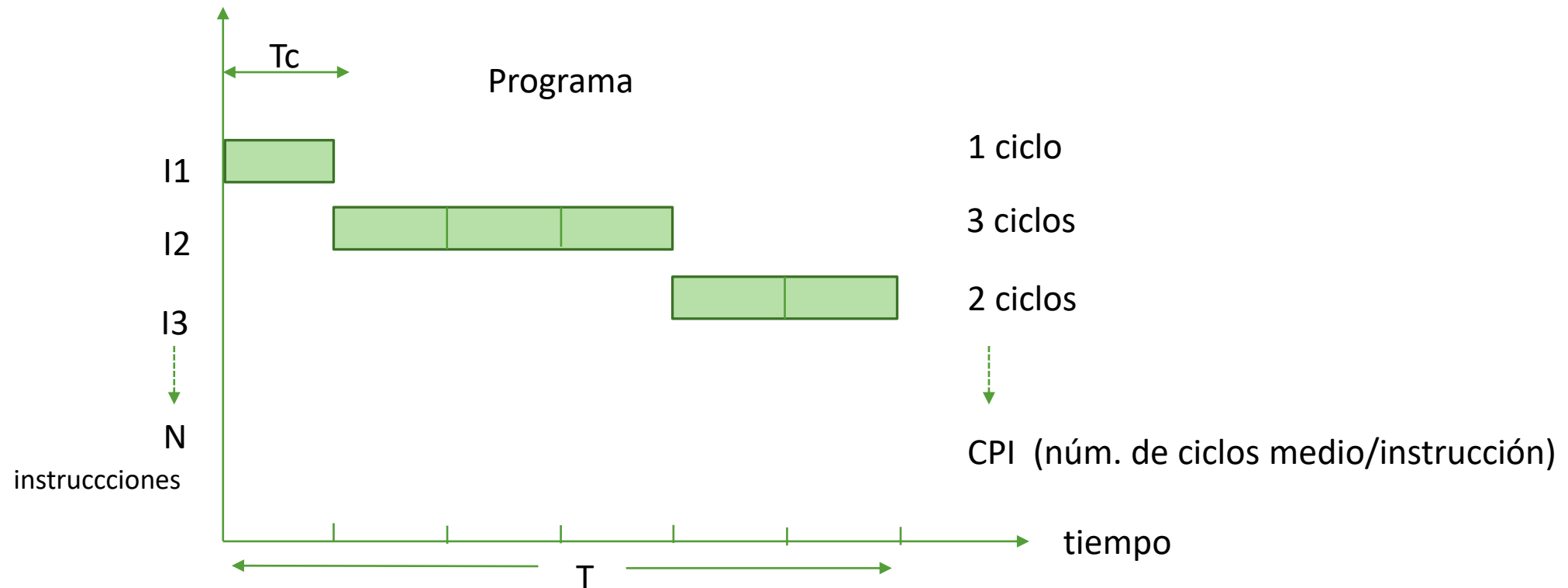
- La **velocidad de procesamiento** de información de un computador está determinada por tres elementos: arquitectura, organización (o estructura) y tecnología.
- Podemos analizar la influencia de estos elementos en la velocidad de procesamiento de información de un computador teniendo en cuenta que **el tiempo de ejecución de un programa** (T) se puede expresar como el producto de tres factores: el número de instrucciones del programa (N), el número medio de ciclos por instrucción (CPI), y el tiempo de ciclo (T_c)

$$T = N \times CPI \times T_c$$

Rendimiento

Computador

$$T = N \times CPI \times T_c$$



Rendimiento

La organización y arquitectura de computadores ha experimentado un desarrollo gradual que ha ensalzado aspectos responsables de las mejoras de rendimiento.

Mejoras de rendimiento

- Paralelismo de datos y funcional
 - Procesadores segmentados y superescalares
 - Procesadores vectoriales
 - Multiprocesadores y multicomputadores

Rendimiento

TOP500

- El **TOP500** es una lista que clasifica a los 500 supercomputadores más potentes del mundo en términos de rendimiento.
 - Se publica dos veces al año (en junio y noviembre) y se basa principalmente en la capacidad de procesamiento de estos sistemas, medida a través del High Performance Linpack (HPL), un benchmark que evalúa el rendimiento de los supercomputadores en la resolución de ecuaciones lineales.
- El objetivo del TOP500 es destacar los avances en el ámbito de la supercomputación y ofrecer un panorama de las **tecnologías más avanzadas y eficientes**.
- Esta lista **no solo mide la velocidad de procesamiento**, sino que también proporciona información sobre las arquitecturas utilizadas (procesadores, redes, etc.), eficiencia energética y las tendencias en supercomputación global).
- Es una referencia importante en el campo del High Performance Computing (HPC), utilizado para investigaciones en física, biología, meteorología, inteligencia artificial, entre otros.

Rendimiento

TOP500 – Junio 2025

- <https://top500.org/>
- Benchmark **Linpack** (HPL).
 - **Rmax** es el rendimiento máximo medido en pruebas prácticas, reflejando lo que el sistema realmente puede lograr en un entorno de trabajo real bajo la prueba Linpack.
 - **Rpeak** es el rendimiento máximo teórico. Representa la cantidad más alta de operaciones que el sistema podría realizar en teoría, si todos los núcleos funcionaran al 100% de su capacidad sin ningún tipo de ineficiencia.
 - PetaFLOPS: 10^{15} (mil billones) de operaciones por segundo.

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

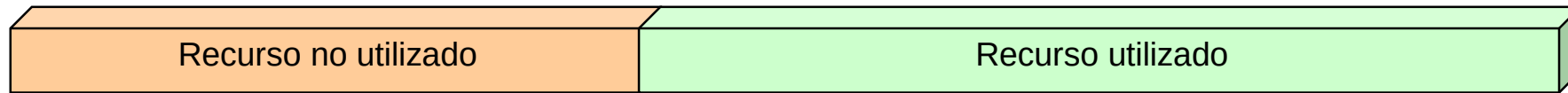
Mejoras en prestaciones

- Gene Amdahl, un arquitecto de las primeras computadoras IBM, formuló su famosa ley para señalar algunas limitaciones del procesamiento paralelo.
- La mejora obtenida en el rendimiento global de un computador al utilizar algún modo de ejecución más rápido está limitada por la fracción de tiempo que se puede utilizar ese modo más rápido.
- La ganancia (G) de velocidad global (también denominada aceleración – Acc) de un sistema se define por el siguiente cociente:
 - T_sin: Tiempo de ejecución sin mejora (Original)
 - T_con: Tiempo de ejecución con mejora (Mejorado)

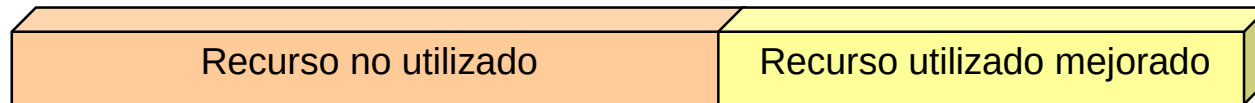
$$G = \frac{T_{\text{sinMejora}}}{T_{\text{conMejora}}} = \frac{T_{\text{original}}}{T_{\text{mejorado}}}$$

Mejoras en prestaciones

$$G = \frac{T_{\text{sinMejora}}}{T_{\text{conMejora}}} = \frac{T_{\text{original}}}{T_{\text{mejorado}}}$$



Tiempo sin mejora

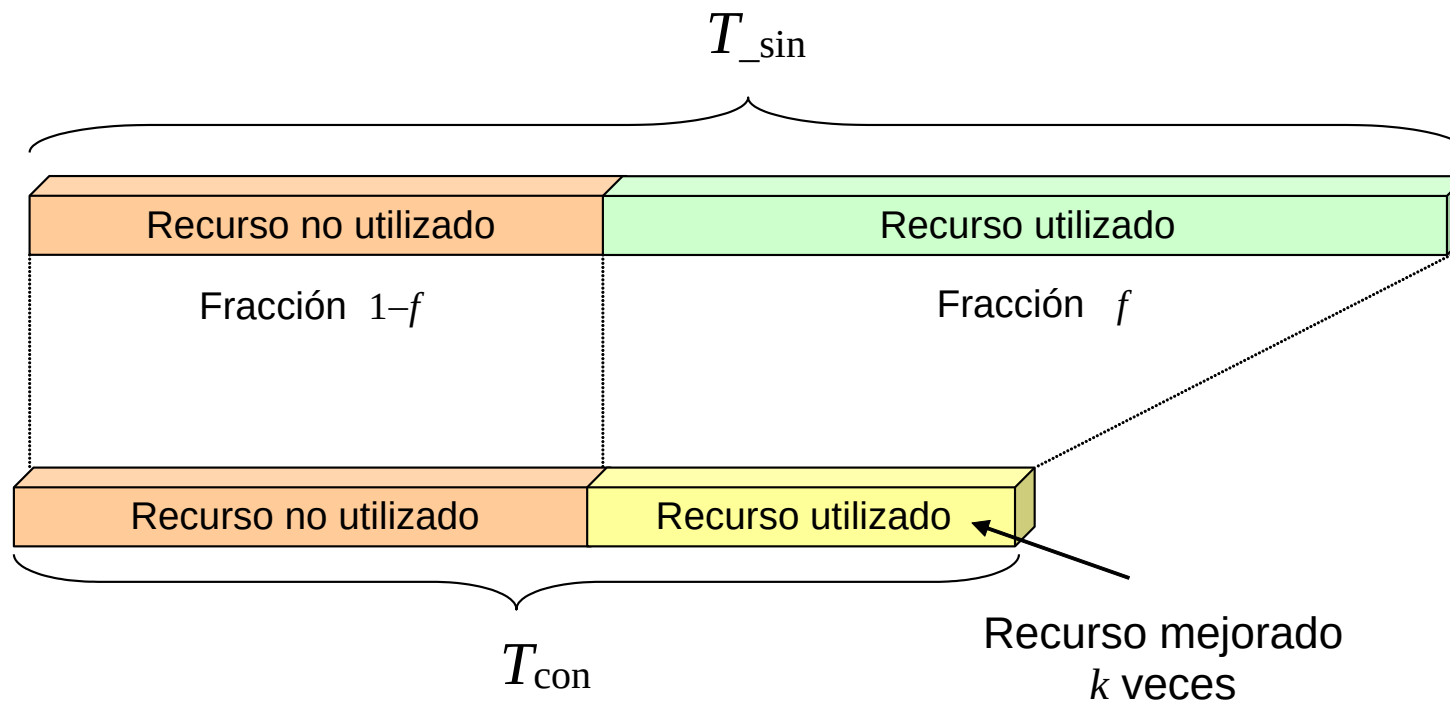


Tiempo con mejora

$$G = \frac{T_{\text{original}}}{T_{\text{mejorado}}} = \frac{T_{\text{recursoUtilizado}}}{T_{\text{recursoUtilizadoMejorado}}}$$

Mejoras en prestaciones

- Si f representa la fracción del tiempo de corrido de un programa debido a tareas que utilizan el recurso mejorado (incluso suponiendo que el resto del programa disfrute de la aceleración perfecta), la aceleración global estaría limitada a la fracción de tiempo que no se utiliza la mejora.



$$G = \frac{T_{sin}}{T_{con}}$$

$$T_{con} = T_{sin} \times \left((1 - f) + \frac{f}{k} \right)$$

$$G = \frac{1}{(1 - f) + \left(\frac{f}{k} \right)}$$

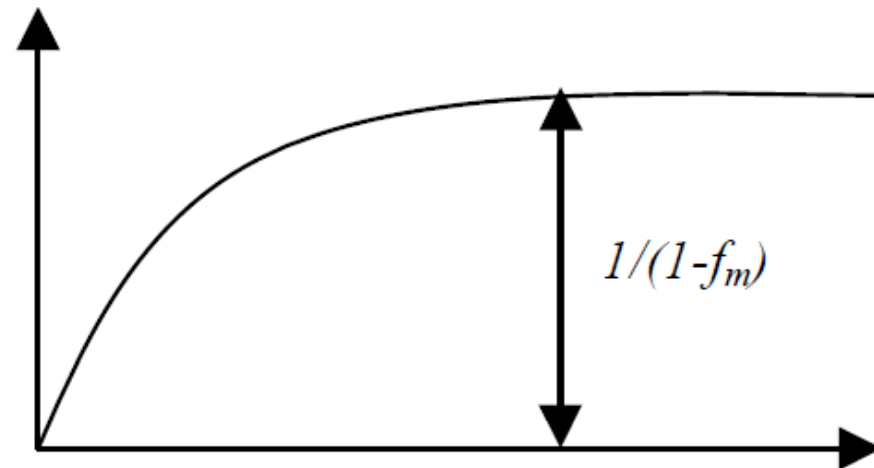
Mejoras en prestaciones

La mejora de velocidad (ganancia – G), que se puede obtener cuando se mejora un recurso de una máquina en un factor k está limitada por la fracción del tiempo de ejecución durante el que no se puede aplicar esa mejora en la máquina.

Ejemplo ilustrativo: un sistema tarda un tiempo en ejecutar un programa (Tiempo original). Mejoramos el sistema acelerando k veces uno de sus componentes. Este componente se utiliza durante una fracción f del tiempo original ¿Cuál es la ganancia del sistema global?

$$\lim_{k \rightarrow \infty} G = \frac{1}{(1-f) + \left(\frac{f}{k}\right)}$$

$$G = \frac{1}{(1-f)}$$



Mejoras en prestaciones

Ejemplo 1. ¿Qué cambio es más efectivo en un procesador determinado?

- A) Acelerar 10 veces únicamente la operación de raíz cuadrada en punto flotante, que ocupa el 20% del tiempo de ejecución
- B) Acelerar 2 veces todas las operaciones en punto flotante, que constituyen el 50% del tiempo total de ejecución?

Nota: se supone que el coste de realizar cualquier cambio es el mismo y que los dos cambios son mutuamente excluyentes.

$$G_{SQ} = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = 1.22, \text{ o } 22\%$$

$$G_{FP} = \frac{1}{(1 - 0.5) + \frac{0.5}{2}} = 1.33, \text{ o } 33\%$$

Solución:

Mejora la unidad de punto flotante (B)

Mejoras en prestaciones

Casos particulares

Se puede señalar que:

- Si $f = 0 \rightarrow G = 1$, ya que no hay ninguna mejora en el sistema.
- Si $f = 1 \rightarrow S = k$, ya que el sistema mejora igual que el componente.

Ejemplo 2. El 10% del tiempo de ejecución de un programa es consumido por operaciones en Punto Flotante (PF). Se mejora la implementación de la operaciones PF reduciendo su tiempo a la mitad ($k=2$) ¿Cuál es el porcentaje de mejora?

$$T_{\text{mejorado}} = T_{\text{original}} \times \left((1 - f) + \frac{f}{k} \right)$$

$$T_{\text{mejorado}} = T_{\text{original}} \times \left((0.90) + \frac{0.10}{2} \right)$$

$$T_{\text{mejorado}} = T_{\text{original}} \times 0.95$$

$$G = \frac{T_{\text{original}}}{T_{\text{mejorado}}} = \frac{T_{\text{original}}}{T_{\text{original}} \times 0.95} = 1.053$$

Solución:

Mejora un 5.3%

Mejoras en prestaciones

Ejemplo 3. ¿Cual es la mejora de la velocidad de una aplicación cuando se ejecuta el 90% del trabajo sobre 100 procesadores en paralelo? El 10% restante no admite la ejecución en paralelo

$$T_{\text{mejorado}} = T_{\text{original}} \times \left((1 - f) + \frac{f}{k} \right)$$

$$T_{\text{mejorado}} = T_{\text{original}} \times \left((0.10) + \frac{0.90}{100} \right)$$

$$T_{\text{mejorado}} = T_{\text{original}} \times 0.109$$

$$S = \frac{T_{\text{original}}}{T_{\text{mejorado}}} = \frac{T_{\text{original}}}{T_{\text{original}} \times 0.109} = 9.17$$

Solución:

El uso de 100 procesadores multiplica la velocidad por 9.17

Mejoras en prestaciones

Ejemplo 4. La utilización de un procesador es del 60% del tiempo total de una aplicación en ejecutarse. ¿En cuánto aumentará el rendimiento del sistema si se duplica la velocidad del procesador ($k=2$)?

$$G = \frac{1}{(1-f) + \left(\frac{f}{k}\right)}$$

$$G = \frac{1}{(1-0.6) + \left(\frac{0.6}{2}\right)} = 1.43$$

Solución:

El rendimiento aumenta un 43%

Ejemplo 5. ¿Cuál es la ganancia máxima que se puede conseguir utilizando el procesador?

$$\lim_{k \rightarrow \infty} G = \frac{1}{(1-f)} = \frac{1}{(1-0.6)} = 2.5$$

Solución:

El rendimiento máximo puede llegar a alcanzar 2.5 veces el rendimiento original

Mejoras en prestaciones

Ejemplo 6. Se tiene un sistema con p procesadores y se quiere paralelizar una aplicación donde el 10% del código es secuencial y el resto es paralelizable a través de 15 procesadores. ¿Cuál es la ganancia en velocidad?

$$G = \frac{1}{(1 - f) + \left(\frac{f}{k}\right)}$$

$$G = \frac{1}{(1 - f_{\text{paralela}}) + \left(\frac{f_{\text{paralela}}}{p}\right)} = \frac{1}{0.10 + \left(\frac{0.90}{15}\right)} = 6.25$$

Solución:

El rendimiento aumenta 6.25 veces con 15 procesadores

Ejemplo 7. ¿Cuál es la ganancia máxima a través de procesadores en el tiempo paralelizable?

$$\lim_{k \rightarrow \infty} G = \frac{1}{(1 - f_{\text{paralela}})} = \frac{1}{0.10} = 10$$

Solución:

La máxima ganancia con dicha mejora llega a ser 10 veces la original

Áreas estratégicas



Áreas estratégicas

Dentro de Arquitectura de Computadores

- *Las áreas estratégicas en la arquitectura de computadores se centran en cómo mejorar el rendimiento, la eficiencia energética, la escalabilidad y la capacidad de manejar tareas complejas.*
- 1. **Paralelismo y Concurrencia.** Paralelismo implica la ejecución simultánea de múltiples procesos, mientras que concurrencia permite que varios procesos progresen de manera intercalada. Esto se ha convertido en un enfoque esencial en el diseño de CPU y GPU para maximizar el rendimiento. Ejemplo: arquitecturas multinúcleo y sistemas distribuidos.
- 2. **Memoria y Jerarquía de Caché.** Optimizar el acceso a la memoria y las jerarquías de caché es clave para reducir los tiempos de latencia y mejorar la eficiencia. Esto incluye tecnologías como memoria no volátil (NVRAM) y mejoras en la latencia de caché.
- 3. **Eficiencia Energética** Con la creciente demanda de poder computacional, el consumo de energía se ha vuelto crítico. Arquitecturas más eficientes permiten realizar más cálculos por unidad de energía. Ejemplo: Procesadores ARM, que destacan en eficiencia energética frente a x86.

Áreas estratégicas

HiPEAC

"High Performance Edge And Cloud computing" es el principal punto de encuentro para la creación de redes, difusión, formación y actividades de colaboración en Europa para investigadores, industria y política relacionados con los sistemas informáticos.

Desafíos intersectoriales en la visión de HiPEAC

- **Eficiencia energética.** Hacer frente a los retos de la programación paralela en procesadores heterogéneos optimizando la transferencia de datos. También aprovechar el potencial de ahorro energético de las nuevas tecnologías
- **Complejidad de sistemas.** Hay que desarrollar herramientas y técnicas para optimizar el rendimiento, asegurando el correcto funcionamiento, todo ello "a escala".
- Investigar nuevas **técnicas de integración** que permiten altos niveles de integración y diferenciación sin el costo de fabricación de vanguardia.
- **Fiabilidad.** Con metodologías y herramientas más poderosas para diseñar e implementar sistemas fiables a un costo razonable.

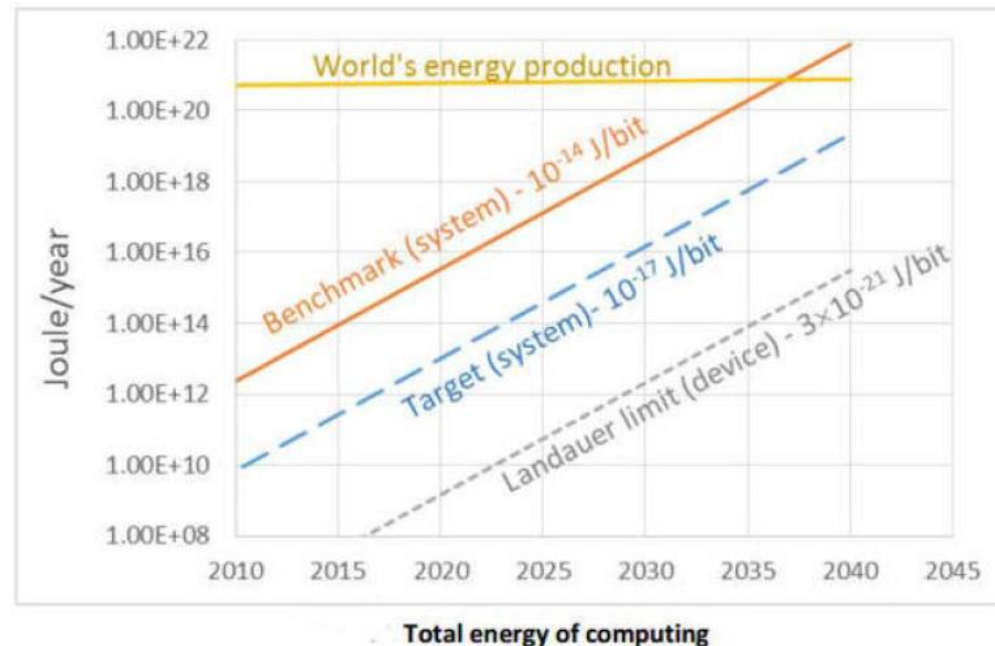
Áreas estratégicas

Un reto transcendental: la eficiencia energética

- La energía se ha convertido en el principal **factor limitante** en el desarrollo de todos los sistemas.
- Importancia del **coste de la energía** para el propio funcionamiento del sistema, el coste de energía para climatización en los sistemas grandes o la vida de la batería en los dispositivos móviles.
- El consumo energético de los programas ha pasado a ser una **medida de prestaciones tan importante como el tiempo de procesamiento**, a pesar de que no suele incluirse en las medidas de rendimiento de los programas.
- La obtención de una **mayor eficiencia energética** ha llevado a la aparición:
 - De **dispositivos paralelos y heterogéneos** que generan mayor complejidad e incompatibilidad con el software existente.
 - El concepto de "**silicio negro**", consistente en que partes de un dispositivo deben estar apagados para mantenerse dentro del límite de potencia.
 - Dispositivos **conscientes** de energía capaces de automatizar la optimización del consumo de energía, duración de las baterías, etc.

Eficiencia Energética

- Un equipo de expertos ha estimado que en el 2040 las máquinas (desde superordenadores a móviles) consumirán más energía de la que se genera a nivel global.
- Teknautas Victor Zhirnov, director científico de la SRC advierte: *"Mientras que la producción de energía mundial ha crecido linealmente, la demanda de electricidad de los ordenadores lo ha hecho de manera exponencial"*.



Eficiencia Energética

Green500 List

- [Junio 2025](#)
- Métrica para **comparar la eficiencia energética** entre computadores:
- Gflops (GigaFlops: 10^9 (mil millones) de operaciones por segundo por vatio (watts)).
- Cuanto mayor sean los Gflops/watts de un procesador, mejor será desde el punto de vista del consumo de energía.

Rank	TOP500 Rank	System	Cores	Rmax (PFlop/s)	Power (kW)	Energy Efficiency (GFlops/watts)
1	259	JEDI - BullSequana XH3000, Grace Hopper Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, ParTec/EVIDEN EuroHPC/FZJ Germany	19,584	4.50	67	72.733
2	148	ROME0-2025 - BullSequana XH3000, Grace Hopper Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, Red Hat Enterprise Linux, EVIDEN ROME0 HPC Center - Champagne-Ardenne France	47,328	9.86	160	70.912
3	484	Adastra 2 - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, RHEL, HPE Grand Equipement National de Calcul Intensif - Centre Informatique National de l'Enseignement Supérieur (GENCI-CINES) France	16,128	2.53	37	69.098
4	183	Isambard-AI phase 1 - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE University of Bristol United Kingdom	34,272	7.42	117	68.835
5	255	Otus (GPU only) - ThinkSystem SD665-N V3, AMD EPYC 9655 96C 2.6GHz, NVIDIA H100 SXM5 80GB, Infiniband NDR, Rocky Linux 9.4, Lenovo Universitaet Paderborn - PC2 Germany	19,440	4.66		68.177

Recursos

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- Arquitectura de Computadores de Julio Ortega, Alberto Prieto y Mancia Anguita. Ediciones Paraninfo – 978849732274
- Evolución de las tecnologías de informática — Visión HiPEAC ([link](#))
- Conferencia Desarrollo y retos de la Arquitectura de Computadores. Alberto Prieto 2019 ([link](#))
- The European Processor Initiative ([link](#))